# Ensemble Models for K-Shot ProtoNets

**Anonymous ACL submission**

## Abstract

Few-shot classification with Prototypical Networks is an established method for cases with limited labelled data, or for expanding classifiable classes within a solution. However, the effects and potential for ensemble models to improve accuracy on text classification is still untapped. To advance the understanding of few-shot text classification using Prototypical Networks and ensemble models, this paper proposes training different architectures comprised of famous text classification neural network techniques and Prototypical Networks. Furthermore, it trains the models on an unclean and cleaned version of the dataset. The observable results show that the ensemble models are often not statistically significantly better than the base ProtoNet. However, some conditions do produce significantly better results on ensemble architectures. The cleaned version of the dataset has significant improvements across the board, but it especially boosts the ensemble models performance. This paper sets the groundwork for future research into the potential benefits of ensemble models for text few-shot binary classification.

## 1 Introduction

Our intuition within the field of AI would tell us that 20 data points is not enough data to train an accurate model, but the field of Few-Shot or k-shot[1] classification has challenged this presupposition by training accurate models using a small number of training data points, both for image(Snell et al., 2017; Hou and Sato, 2022; Ji et al., 2020) and text(Sun et al., 2019; Guo et al., 2022; Sourati et al., 2023; Liu et al., 2025) and even for both(Pahde et al., 2021). One technique which stands effective is the Prototypical Network.

For more in-depth information about ProtoNets, refer to the paper that introduces it(Snell et al., 2017). This approach to few-shot classification has demonstrated positive results for image(Snell et al., 2017; Hou and Sato, 2022; Ji et al., 2020) and text classification(Sun et al., 2019; Guo et al., 2022; Sourati et al., 2023; Liu et al., 2025). It is a useful classifier for when there is a limited amount of labelled data or use cases for which there is need to easily add unseen classes in the future.

But the literature rarely attempts to ensemble other models onto it, and its validation sets are comprised of simple use cases. In the real world, we often handle data of complex language and great length. This paper asks how much could a Prototypical Network model improve if assembled with other popular techniques, applied on a complex corpus. One could posit that more complex architectures could potentially improve the models ability to find relations within n-grams and between sentences more effectively. This paper will attempt to prove the null hypothesis; that more complex models affect the accuracy of a Prototypical Network. It will also be exploring the effect of processing the data in a manner of which n-grams are always relevant to the corpus(Yun-tao et al., 2005; Kuang and Xu, 2010). However, complex architectures may potentially not impact, and perhaps penalise, accuracy. Whether due to noise introduced by transposition layers or because of overfitting, complex models can indeed make things worse. When exploring ensemble models, one must assume this to be the null hypothesis, and have a reason to challenge it. Throughout the literature that introduces novel methods for text classification, it is standard to use known datasets for validation, like FewRel(Sun et al., 2019) or IMDB reviews(Sourati et al., 2023), whose corpus often lacks the complexity found in the academic sphere. We can take as evidence for this challenge the generated tags attributed to projects in the HORIZON portal. These often are inaccurate or lacking. And although their methods are unknown, one can hypothesise that

[1] k-shot classification where k is the number of labelled examples a given model is trained on

the complexity of the sentence structure and vocabulary will affect any NLP and word-vector based task. Therefore, exploring the effect of more complex architectures, which are proven to be capable of capturing n-gram and cross-sentence relations, would arguably be a valid question. This paper aims to provide evidence that these complex architectures are worth investigating, by means of testing their performance on HORIZON projects, which feature PhD levels of corpus complexity (see Figure 2).

The conducted research also introduces a new technique within NLP and RNNs called xLSTM(Beck et al., 2024), which shows promising results for NLP and vastly improve on the commonly used for classification LSTM(Sachan et al., 2019; Xiong et al., 2024; Liu et al., 2024; Kaur and Kaur, 2023; Jamshidi et al., 2024; Zhou et al., 2015). With the introduction of ensemble models into the Prototypical Network, and the use of novel methods like xLSTM, this paper situates itself within the field of few-shot text classification, and ensemble architecture modelling. The lack of studies on ensemble models in Prototypical Networks is a research gap that this study aims to contribute in its filling.

## 2 Methods

For the purposes of this research, I used a dataset of labelled projects from the publicly available EU CORDIS research database[2]. In order to methodically explore the possibilities of expansion, it prompts to be addressed the complexity of the dataset. Our dataset contains complex academic jargon, compared to the more common lexicon in most public validation datasets used in the literature. [3] This means that the text used through this paper will be more complex than the datasets utilised in most literature on Prototypical Networks for text classification(Sun et al., 2019; Guo et al., 2022; Sourati et al., 2023). To tend for this contrast, there will be a trial without and with preprocessing of the projects descriptions, henceforth known as the raw and processed version of the dataset. Therefore, some parts of the preprocessing sub-section of this chapter will only apply to the trial in which the descriptions have been preprocessed. This distinction shall be made obvious.

[2]The raw data was downloaded in October 2024. The current data is available at data.cordis.eu. The labelled data used in this paper is available at solid-sustainability.org/ccus-tables, solid-sustainability.org/h2-tables, and solid-sustainability.org/solar-tables.

[3]By simple task, it is meant to signify a classification task on which the text data uses common sentence structures and vocabulary e.g. sentiment analysis of short sentences, relational, etc. This distinction is made because the descriptions of the projects are often filled with academic jargon and complex sentence structures.

| label | count |
|---|---|
| NL | 118027 |
| solar,solarPlus | 1779 |
| H2 | 915 |
| CCUS | 518 |
| solarPlus | 357 |
| H2,solar,solarPlus | 114 |
| CCUS,H2 | 49 |
| CCUS,solar,solarPlus | 41 |
| CCUS,H2,solar,solarPlus | 8 |
| H2,solarPlus | 2 |

Table 1: Number of projects per label combination

### 2.1 Data

#### 2.1.1 Characterisation

The dataset is a combination of multiple datasets that were previously divided by their labels, of which are solar, solarPlus, hydrogen (H2) and Carbon Capture Utilisation and Storage (CCUS). Solar is a subset of solarPlus. This was a decision by the researchers that labelled the dataset. These will be our classes. One project may be attributed with multiple labels. The dataset includes 121808 distinct projects, with their title, country, fossil partner (if relevant), start and end date and the label given by the researchers. Most importantly, they also contain a description of the project, which will be the singular feature on which the models will be trained.

The biggest class is NL (n = 118027) which stands for Not Labelled, which will be treated as 0 in the binary classification. The biggest distinct class group is solar and solarPlus (n = 1779), followed by H2 (n = 915). The other distinct classes are CCUS (n = 518), solarPlus (n = 357).

Additionally, it is observable in Table 1 that the solar category never appears ungrouped. That is because solar is a subclass of solarPlus, according to the researchers that set the labels and carried out the labelling. Later on, all classes will be ungrouped, meaning that projects with multiple classes might appear in more than one dataset as a labelled data point.

The dataset for a given class is comprised of all

| class | AW rate | LFW rate | count | Avg word count | Median word count |
|---|---|---|---|---|---|
| CCUS | 21.3 | 5.3 | 616 | 267 | 276 |
| H2 | 21.8 | 4.6 | 1088 | 271 | 276 |
| solar | 21.1 | 4.5 | 1942 | 274 | 274 |
| solarPlus | 21.1 | 4.5 | 2301 | 277 | 275 |
| NL | 20.5 | 4.7 | 118027 | 277 | 274 |

Table 2: Analysis of corpus complexity per label

of the data entries with that specific label, and all the other data entries of all the other classes plus the data entries with no label (NL = Not Labelled).

It is important to highlight how the average and median word count is pretty similar for all labels. This denotes that differences in accuracy that might occur on a dataset basis are not likely to be due to differences in text length.

The complexity algorithm was calculated by the ratio of AW (Academic Words) and LFW (low frequency words)(Ferlatti). According to the Word Complexity ratio, all of the classes showcase a corpus at PHD level (ratio =< 5), which means that differences in accuracy may not be derived from complexity. For more details on how this ratio was calculated, refer to the cited paper.

However, it is worth noting that the CCUS class features a significantly higher rate of low frequency words than the rest (*p-value* $< 0.05, p \approx 0.0002$). Contrastly, solarPlus has a significantly lower rate of LFW (*p-value* $< 0.05, p \approx 0.007$). For AW, H2 uses significantly more academic jargon than the rest (*p-value* $< 0.05, p \approx 0.03$). On the other hand, NL has significantly less academic jargon in its corpus (*p-value* $< 0.05, p \approx 0.006$)

#### 2.1.2 Preprocessing

The dataset containing all the data is then separated into individual datasets where the projects with a given class are assigned a 1 and all the other projects are assigned a 0. This way, a project spanning multiple classes is included in the file for those classes as 1.

There were two experiments carried with different versions of the same datasets. The Unclean version of the dataset will simply feature the datasets for each label containing only that designation and its untreated project description.

Another version of those individual datasets, henceforth known as the Cleaned datasets, descriptions of the project were processed by using tf—idf(Kuang and Xu, 2010; Yun-tao et al., 2005) with a weighted approach, where only the most important words (top k = 3000) are included. The features are extracted from the aggregate dataset, so as to capture not only key words relevant to a class, but also to include slang typical to academic text. The description of the projects, thus, becomes only a string of the most important words, ergo the most important words for a domain will repeat multiple times.

With this, we get a dataset that is ready to be segmented and that can then be manipulated to achieve batches for fitting.

#### 2.1.3 Preparation

The datasets undergo a 80/20 split. The split is done once and remains constant throughout all experiments, to ensure in latter stages that the difference between accuracy is due to the changes in model architecture and weight initialisation only. This is done for all datasets.

Training and validation sets are tokenised using BERT base uncased(Devlin et al., 2019) due to its lightweight, yet capable and wildly available proportions.

Random samples of the dataset are picked. ***k*** number of examples for each class are drawn. Using those samples, a number of examples are derived from ***k*** plus the query number (n=15), with which is selected from the general dataset that number of examples for each distinct class at random.

### 2.2 Architectures

This paper attempts to experiment with ensemble models, in order to capture the complexity of the dataset. The hypothesis tested is that the complexity of the dataset should be dealt with a proportionately complex model architecture. LSTMs(Jamshidi et al., 2024; Liu et al., 2024; Sachan et al., 2019), CNNs(Kim, 2014) and their ensemble(Zhou et al., 2015; Kaur and Kaur, 2023; Xiong et al., 2024) have been shown to greatly improve performance in text classification, even

when the body of text is of a complex nature(Liu et al., 2024; Xiong et al., 2024). xLSTM is a new architecture which further improves on the accuracy of BiLSTMs(Beck et al., 2024). Therefore, it is a natural inclusion to ensemble onto the base ProtoNet.

All models have hidden dimensions of 256 except for ProtoCNNBiLSTM, which has double of that, being 512 because BiLSTMs are, in simple terms, two layers of LSTM stacked.

Euclidean distance was chosen for being simple to implement. It outperforms cosine similarity in a comparible study (Snell et al., 2017), and based on pilot tests, is adequate to test these models.

All architectures encode the embeddings in different ways, but they all feed their output to the mechanism that calculates the Prototypical Networks. CNNs are used to capture n-gram level patterns while the LSTM layers focuses on temporal patterns (think of the order of each word in a sentence as its temporal feature).

There are two architectures which contain a new technique called xLSTM(Beck et al., 2024). By modifying the LSTM memory structure and introducing exponential gating, the model amplifies the importance of the differences between similar words during training. When paired with CNN, the xLSTM layer operates on its own separate branch and then fussed with the output of the CNN.

### 2.3 Experiment

Each class will have its own binary classification model.

For each architecture and class, a model is fitted for 35 epochs, with 15 support queries. For each k-shot configuration, which is either 2, 3, 5, 10 or 50-shot, the model configuration is fitted 5 times for each model and class. The same experiment is run 5 times to account for deviations in weight initiation. For example, for 5-shot, all the models would be fitted 5 times for all class datasets.

This format is done for both the uncleaned datasets and their processed version.

The experiment uses Adam optimiser ($lr = 1e-3$).

For validation, accuracy, precision, recall and F1 scores are collected, along with p-score.

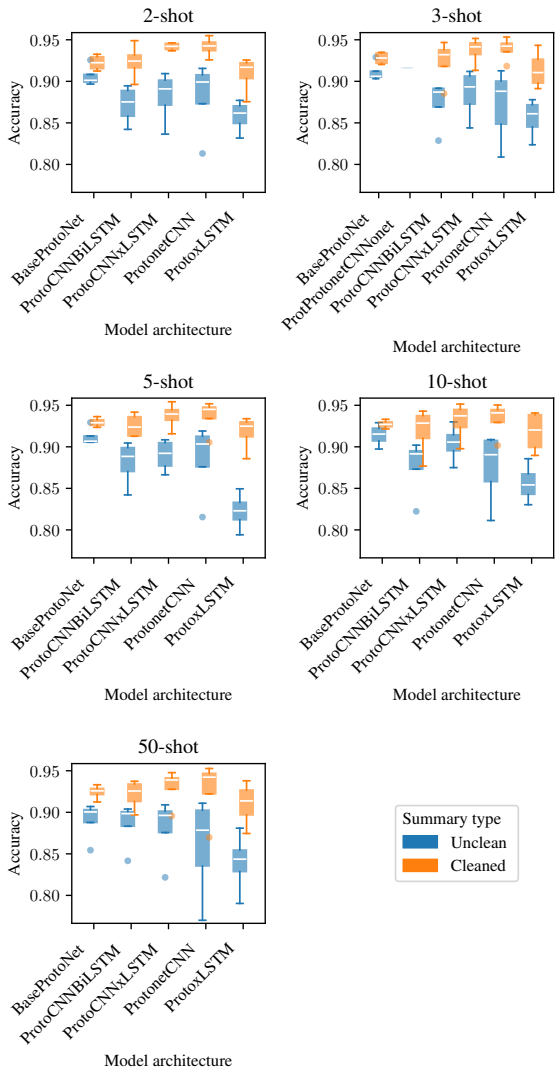


Figure 1: Aggregate results per k-shot between architectures and preprocessing methods

## 3 Results

As seen in Figure 1, there is no significance ($p > 0.05$) in accuracy[4] between k-shots. It is however abundantly clear that the difference in accuracy between the Unclean and Preprocessed (Cleaned) datasets is significant ($p = 0.000, p < 0.001$), being on average $\approx 5.13\%$ more accurate. The best performing architecture on the unclean set is the Base Prototypical Network. It also is the most consistent in its results, showing less deviation and outliers from the mean. The overall picture changes when we consider the cleaned set, which features

[4]The reason why this paper only reports accuracy is because, due to the way the results are validated, which is balanced perfectly between 0 and 1 class, the difference between metrics is negligible between accuracy and all other common validation metrics. Therefore, we assume accuracy in its simplest form.

the Protonet + CNN and the Protonet + CNN + xLSTM as the best models. However, the Base Prototypical Network continues to be the most stable architecture. ProtoNetCNN is the best performing model ($\approx 95\%$ mean accuracy) for 2, 3 and 50-shot. However, for 5 and 10-shot, ProtoCNNxLSTM performs better overall ($\approx 95\%$ mean accuracy). However, these results per k-shot are not statistically significant ($p > 0.05$). These results do not account for variations between classes.

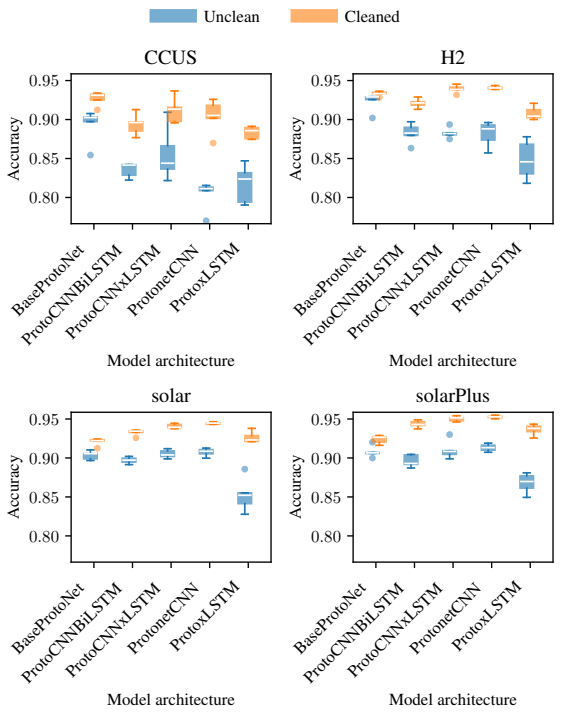


Figure 2: Aggregate results per class between architectures and preprocessing methods

Figure 2 shows the same results but per class. The dataset which the models struggled the most with was CCUS, where the best performing architecture was the Base Prototypical Network ($\approx 91\%$), significantly higher than the other architectures (except for ProtoCNNxLSTM and ProtoNetCNN) ($p < 0.05$). It was also on this class that the results showed greater variation. For all classes, the cleaned set continues to outperform the unclean. Concerning the H2 class, ProtoCNNxLSTM, ProtoNetCNN and BaseProtonet are not significantly different from each other ($p > 0.1$). For solar, only ProtoNetCNN and ProtoCNNxLSTM performed significantly better than the rest ($p < 0.05$). Lastly, the solarPlus class is most accurately predicted by the ProtoCNNxLSTM and ProtoNetCNN architectures, which are significantly

| Dataset | k-shot | Model | Mean Accuracy |
|---|---|---|---|
| CCUS | 2 | BaseProtoNet | **93.67%** |
| | | ProtoCNNxLSTM | 93.26% |
| | | ProtoNetCNN | 92.59% |
| H2 | 2 | BaseProtoNet | **94.54%** |
| | | ProtoCNNxLSTM | 94.0% |
| | | ProtoNetCNN | 92.85% |
| solarPlus | 2 | ProtoCNNBiLSTM | **95.49%** |
| | | ProtoCNNxLSTM | 94.89% |
| | | ProtoNetCNN | 94.62% |
| | | ProtoxLSTM | 92.56% |
| solar | 5 | ProtoCNNBiLSTM | **94.65%** |
| | | ProtoCNNxLSTM | 93.75% |
| | | ProtoNetCNN | 93.44% |

Table 3: Best performing model configuration for each dataset

better than the rest ($p < 0.05$). These results do not account for variations caused by different k-shots.

For CCUS, H2 and solarPlus, the best k-shot configuration is 2-shot. The best models are BaseProtoNet, ProtoCNNxLSTM and ProtoNetCNN. Additionally, solarPlus also features ProtoCNNBiLSTM. For solar, the best k-shot configuration is 5-shot, and the best performing models are ProtoCNNBiLSTM, ProtoCNNxLSTM and ProtoNetCNN. All of these 'best models' mentioned are not significantly different between each other ($p > 0.05$).

## 4 Discussion

If the objective is to evaluate the viability of Ensemble Architectures with a Prototypical Network base, then the results are inconclusive. In some datasets, a more complex model is detrimental to the accuracy of the task, as shown in Figure 2, where the uncleaned dataset clearly shows worst performance, except for solar and solarPlus, where it is not statistically significant. So, contrary to the initial hypothesis that more complex models would perform better in more complex and longer texts, it looks as though the more complex models perform worse in such conditions.

However, the cleaned versions of the datasets seem to reverse this trend somewhat. That the accuracy would increase as noisy and unimportant words are filtered out is to be expected (accounting for a $\approx 5.13\%$ increase in accuracy). But for the complex models to not be significantly worse than the BaseProtoNet is a suprise and of relevance. In fact, as seen on Table 3, the BaseProtoNet performs significantly worse than all models in solar and solarPlus, albeit on 5-shot, where the best re-

sult was recorded. On the other two datasets, it is not significantly better. Figure 2 reinforces this observation. The implication of this is in that it subverts the expectation that a more complex model is better when there is less noise in the corpus. It is also worthy of note that, as represented in Table 2, project descriptions labelled with CCUS have significantly more low frequency words in its vocabulary, as well as having a high academic word count. It is also the dataset where the BaseProtoNet performed best. The dataset for H2, which has the highest amount of academic jargon, is where the BaseProtoNet also ranks amongst the most accurate. Although this study cannot prove that there is a correlation between the complexity and even length of the corpus, and the difference in accuracy between the BaseProtoNet and the more complex models, one could speculate that the BaseProtoNet is better at singling out and extrapolating from outlier words to create the prototypes, while the more complex models perform better when the relations between the words present at the corpus is more important than its jargon. This hypothesis is further supported by the literature which suggests that LSTM's in text classification have the function of sequence capturing(Xue et al., 2022), while CNN's improve on intersentence pattern recognition(Zhou et al., 2015). But both are good to capture the contextual relation between the training data(Kaur and Kaur, 2023). However, the difference between accuracy variation by model might not be due to the difference in average complexity. So, for future work, it would be decissive to know if there is a correlation between complexity of the corpus and complexity of the model architecture. This exploration could potentially look into amplifying complex words, and thus, improve accuracy. By attempting to prove this hypothesis, one would also be validating the hypothesis that complex models perform better with complex text.

For what concerns the relation between K-shot and accuracy, although one might be tempted to look at Figure 3 and draw conclusions about the effect of k-shot and accuracy, it is worth noting that the difference between the max and minimum accuracy for each model is never statistically significant ($p > 0.4$). Looking at Figure 1, there doesnt seem to be a significant inter-model discrepancy between k-shots. It is however interesting how the ProtoNetCNN and ProtoCNNxLSTM peak on 2-shot, while the others have a more accute variation. The line in ProtoNetCNN is even linear. This may

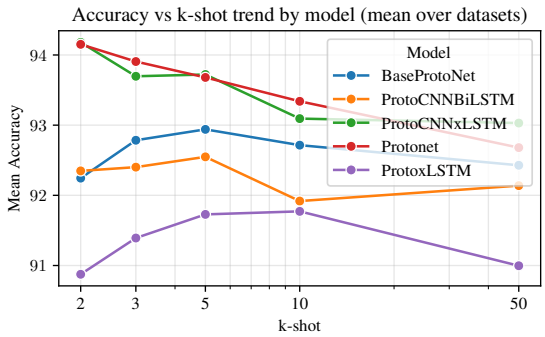


Figure 3: Correlation between accuracy per model and K-shot

be relevant because, in constrast wit the BaseProtoNet, which peaks in 5-shot, the introduction of the CNN component might impact the training in a way which makes it beneficial to have a lower k-shot. More work needs to be done to understand why changing the k-shot is not statistically significant, and smaller k-shots seem to perform better, for one would suppose the more data disposable to the model during training, the better it would perform. One could speculate that, on the datasets where the lowest k-shot performed best, the corpus is more complex and the BaseProtoNet is the best model. Conversly, the k-shot that performed best for solar (a less complex dataset) is higher, reminiscing back the idea that in this dataset, the relation between the words and sentences are more important than the complexity of the lexicon, as discussed in the Methods section of this paper. The results on table 3 might also be outliers within their own configuration. Figure 1 would lead us to conclude such. Variations in model initialisation in non-deterministic models such as neural networks might mean that the best performing models coming out of the experiments might have slight variations. This, too, is not proven in the scope of this research, and could open the avenue for further research on the variation that the non-deterministic factor has when observing the best performing models. Another potential research venture would be to take the idea of an ensemble model with ProtoNets but apply processing to turn the text of the descriptions into image features, and then combine the ProtoNet and a CNN, which has been proven to work for text classification(Gallo et al., 2017). Or even using Fourier Transform and the Transformer architectures for the same purpose(Kachare et al., 2025). Since CNNs are traditionally used

for Image Classificaton, these suggestions seem relevant because the introduction of the CNN architecture performed remarkably well in these experiments. To further the understanding of why CNN improves accuracy when combined with ProtoNets is essential not only to further our understanding of few-shot image classification; building upon the original paper while exploring novel ways to train few-shot text classification models. To conclude this section, it remains inconclusive whether or not Ensemble models improve or penalise the accuracy of a model. There is too much variation between datasets, models and k-shots. Therefore, future works should aim to understand when a corpus benifits from more complex models, and if so, what is it in that model that improves the accuracy.

## 5 Conclusion

This paper presents experiments carried out on labelled classes of HORIZON projects, that explore the gap in Few-shot Text Classification using Prototypical Networks and Ensemble Models, such as CNN, BiLSTM and xLSTM. For the distinct classes, 4 models were fitted mulitple times, on all 4 distinct architectures, with a clean and uncleaned version. While the results from this experiment could not conclude that complex ensemble architectures combined with a Prototypical Network significantly outperform the base Prototypical Network, they show that more complex models do not necessarily perform better on complex data, as we might expect them to, and they open up many new questions and avenues for further research. Despite the BaseProtoNet still being the best for two datasets, it is not significantly better than the accuracy of the ProtoCNNxLSTM and ProtoNetCNN. This paper reinforces the efficacy of Prototypical Networks for few-shot classification while demonstrating the effects of more complex model architectures; their pitfalls and successes. More research needs to be done to widen the scope of these methods and experiments in other applications.

## 6 Limitations

The data provided by Solid Sustainability Research was labelled manually, with the help of ChatGPT's API, with clear guidelines on how they were labelled. (Pereira, 2025) Therefore, there might be some mislabelled projects.

## 7 Ethical considerations

This research was born out of concerns surrounding the ethical implications of Large Language Models, and the curiosity to endeavour an alternative. The context that the dataset is placed in also aims to have positive societal impact, by means of exposing the extent to which universities and other public institutions have ties with the fossil fuel industry.

## References


Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. 2024. xlstm: Extended long short-term memory. *Preprint*, arXiv:2405.04517.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. *Preprint*, arXiv:1810.04805.

Aldo Ferlatti. Estimation of text complexity.

Ignazio Gallo, Shah Nawaz, and Alessandro Calefati. 2017. Semantic text encoding for text classification using convolutional neural networks. In *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, volume 05, pages 16–21.

Xinyu Guo, Bingjie Tian, and Xuedong Tian. 2022. Hfgnn-proto: Hesitant fuzzy graph neural network-based prototypical network for few-shot text classification. *Electronics*, 11(15).

Mingcheng Hou and Issei Sato. 2022. A closer look at prototype classifier for few-shot image classification. *Preprint*, arXiv:2110.05076.

Saman Jamshidi, Mahin Mohammadi, Saeed Bagheri, Hamid Esmaeili Najafabadi, Alireza Rezvanian, Mehdi Gheisari, Mustafa Ghaderzadeh, Amir Shahab Shahabi, and Zongda Wu. 2024. Effective text classification using bert, mtm lstm, and dt. *Data and Knowledge Engineering*, 151:102306.

Zhong Ji, Xingliang Chai, Yunlong Yu, Yanwei Pang, and Zhongfei Zhang. 2020. Improved prototypical networks for few-shot learning. *Pattern Recognition Letters*, 140:81–87.

Atul Kachare, Chandrashekhar Goswami, Ashutosh Gupta, and D.S. Chouhan. 2025. Fnet-gpt: Fourier-based lightweight transformer for emotion-aware text generation using gpt. *Future Technology*, 4(4):138–145.

Kamaljit Kaur and Parminder Kaur. 2023. Improving bert model for requirements classification by bidirectional lstm-cnn deep model. *Computers and Electrical Engineering*, 108:108699.

Yoon Kim. 2014. Convolutional neural networks for sentence classification. *Preprint*, arXiv:1408.5882.

Qiaoyan Kuang and Xiaoming Xu. 2010. Improvement and application of tf•idf method based on text classification. In *2010 International Conference on Internet Technology and Applications*, pages 1–4.

Bingyao Liu, Jiajing Chen, Rui Wang, Junming Huang, Yuanshuai Luo, and Jianjun Wei. 2024. Optimizing news text classification with bi-lstm and attention mechanism for efficient data processing. *Preprint*, arXiv:2409.15576.

Xinyue Liu, Yunlong Gao, Linlin Zong, Wenxin Liang, and Bo Xu. 2025. Guiding prototype networks with label semantics for few-shot text classification. *Pattern Recognit.*, 164(111497):111497.

Frederik Pahde, Mihai Puscas, Tassilo Klein, and Moin Nabi. 2021. Multimodal prototypical networks for few-shot learning. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2644–2653.

Aaron Pereira. 2025. Classifying research project themes in the ec cordis database with the help of llms. `http://solid-sustainability.org/topic-classification`. Solid Sustainability Research, published 16 March 2025.

Devendra Singh Sachan, Manzil Zaheer, and Ruslan Salakhutdinov. 2019. Revisiting lstm networks for semi-supervised text classification via mixed objective function. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01).

Jake Snell, Kevin Swersky, and Richard Zemel. 2017. Prototypical networks for few-shot learning.

Zhivar Sourati, Darshan Deshpande, Filip Ilievski, Kiril Gashteovski, and Sascha Saralajew. 2023. Robust text classification: Analyzing Prototype-Based networks.

Shengli Sun, Qingfeng Sun, Kevin Zhou, and Tengchao Lv. 2019. Hierarchical attention prototypical networks for few-shot text classification. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 476–485, Hong Kong, China. Association for Computational Linguistics.

Yunpeng Xiong, Guolian Chen, and Junkuo Cao. 2024. Research on public service request text classification based on bert-bilstm-cnn feature fusion. *Applied Sciences*, 14(14).

Bingxin Xue, Cui Zhu, Xuan Wang, and Wenjun Zhu. 2022. The study on the text classification based on graph convolutional network and BiLSTM. In *Proceedings of the 8th International Conference on Computing and Artificial Intelligence*, New York, NY, USA. ACM.

Zhang Yun-tao, Gong Ling, and Wang Yong-cheng. 2005. An improved TF-IDF approach for text classification. *J. Zheijang Univ.-Sci. A*, 6(1):49–55.

Chunting Zhou, Chonglin Sun, Zhiyuan Liu, and Francis C. M. Lau. 2015. A c-lstm neural network for text classification. *Preprint*, arXiv:1511.08630.